

9.OPTIMUM GLOBAL THRESHOLDING USING OTSU'S METHOD

AIM :

To perform optimum global thresholding using Otsu's method for image segmentation.

INTRODUCTION:

Thresholding is one of the simplest and most effective techniques in image segmentation. The idea is to classify the pixels of an image into two or more classes based on their intensity levels.

In statistical decision theory, thresholding can be viewed as a classification problem where the objective is to minimize the probability of error in assigning pixels to different classes. Although the Bayes decision rule gives an optimum solution, it requires the probability density functions (PDFs) of the classes, which are often difficult to estimate.

Otsu's method, proposed by Nobuyuki Otsu (1979), provides an elegant and efficient solution for global thresholding. It determines the threshold by maximizing the between-class variance, which ensures that the chosen threshold separates the foreground and background as distinctly as possible.

Otsu's Algorithm may be summarized as follows:

1. Compute the normalized histogram of the input image. Denote the components of the histogram by $p(i)$, $i = 0, 1, 2, \dots, L-1$.
2. Compute the cumulative sums, $P1(k)$, for $k = 0, 1, 2, \dots, L-1$
3. Compute the cumulative means, $m(k)$, for $k = 0, 1, 2, \dots, L-1$
4. Compute the global mean, $\bar{\mu}$,
5. Compute the between-class variance term, $\sigma_B^2(k)$, for $k = 0, 1, 2, \dots, L-1$
6. Obtain the Otsu threshold, k^* , as the value of k for which $\sigma_B^2(k)$ is maximum. If the maximum is not unique, obtain k^* by averaging the values of k corresponding to the various maxima detected.
7. Compute the global variance, σ_G^2 , using Eq. (10-58), and then obtain the separability measure, η^* , by evaluating Eq. (10-61) with $k = k^*$.

CODE: OTSU'S GLOBAL THRESHOLDING

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img=cv2.imread(r"C:\Users\DELL\Pictures\IPfolder\IPLAB_2025\DIP3E_Original
_Images_CH10\Fig1039(a)(polymersomes).tif", cv2.IMREAD_GRAYSCALE)
```

```

hist, _ = np.histogram(img.ravel(), bins=256, range=(0, 256))
p = hist / hist.sum()
L = 256
k_values = np.arange(L)
P = np.cumsum(p)
m = np.cumsum(p * k_values)
mG = m[-1]

sigma_B2 = (mG * P - m) ** 2 / (P * (1 - P) + 1e-10)
max_val = np.max(sigma_B2)
k_candidates = np.where(sigma_B2 == max_val)[0]
k_star = int(np.mean(k_candidates))

print("Otsu Threshold (k*):", k_star)
sigma_G2 = np.sum(((k_values - mG) ** 2) * p)
eta_star = max_val / sigma_G2
print("Separability measure ( $\eta^*$ ):", eta_star)
binary_img = np.zeros_like(img)
binary_img[img > k_star] = 255

plt.figure(figsize=(20,6))
plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")
plt.subplot(1,3,2)
plt.plot(hist)
plt.axvline(x=k_star, color='r', linestyle='--')
plt.title("Histogram & Otsu Threshold")
plt.subplot(1,3,3)
plt.imshow(binary_img, cmap='gray')
plt.title("Otsu Thresholding")
plt.axis("off")

```

OUTPUT:

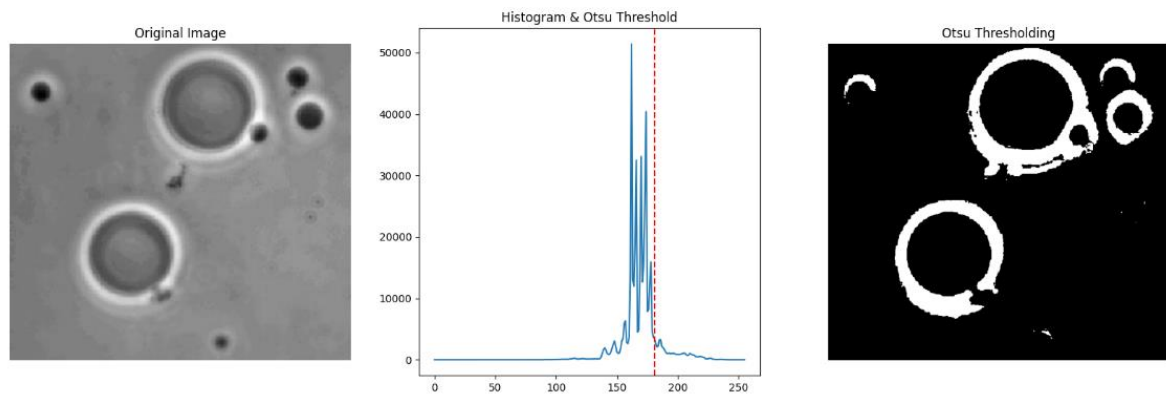


Figure: (a) Original grayscale image (b) Histogram of intensity values with the Otsu optimum threshold marked (red line) (c) Segmented binary image obtained using Otsu's global thresholding, clearly separating objects from the background.

CODE: LOCAL THRESHOLDING

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IPfolder\IPLAB_2025\DIP3E_Original_I
mages_CH10\Fig1039(a)(polymersomes).tif", cv2.IMREAD_GRAYSCALE)

h, w = a.shape
g = np.zeros((h, w), dtype=np.uint8)
for x in range(0, h, 60):
    for y in range(0, w, 60):
        seg = a[x:x+60, y:y+60]
        t = np.mean(seg)
        dt = 1
        while True:
            g1 = seg[seg > t]
            g2 = seg[seg <= t]
            m1 = np.mean(g1) if g1.size > 0 else 0
            m2 = np.mean(g2) if g2.size > 0 else 0

            newt = 0.5 * (m1 + m2)
```

```

        if abs(newt - t) < dt:
            break
        t = newt
    seg_bin = (seg >= t).astype(np.uint8) * 255
    g[x:x+60, y:y+60] = seg_bin

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(a, cmap='gray')
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("Local Thresholding")
plt.imshow(g, cmap='gray')
plt.axis("off")
plt.show()

```

OUTPUT:

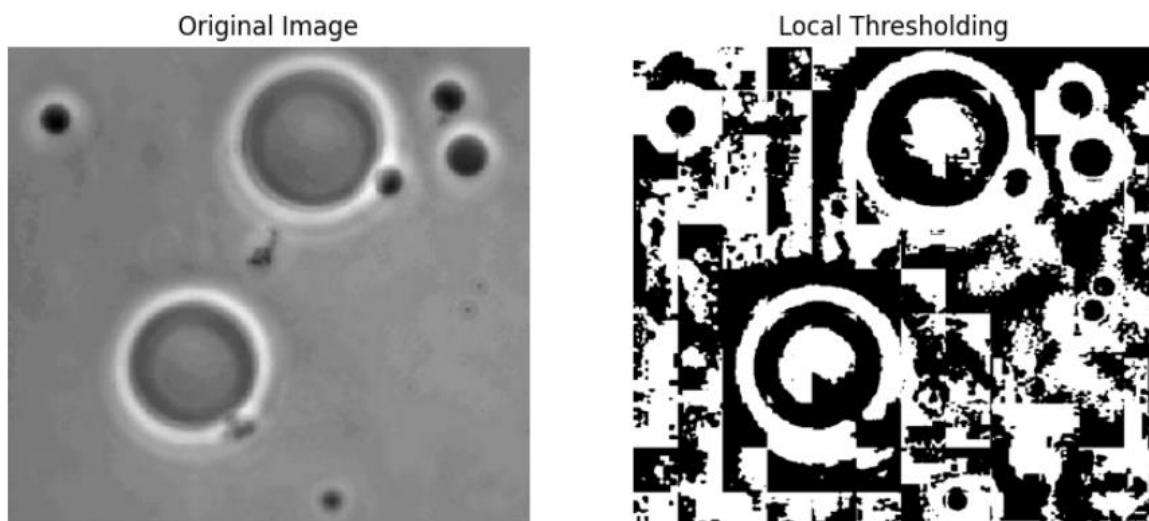


Figure: (a) Original grayscale image with circular objects. (b) Binary image using local thresholding, showing objects segmented but with noisy background and fragmentation.

INFERENCE:

- **Otsu's Global Thresholding** automatically finds the best threshold to separate foreground and background by maximizing the difference between classes. It works well for images with clear contrast and uniform lighting, giving clean and clear segmentation.
 - **Local Thresholding** calculates thresholds for small parts of the image, handling uneven lighting better.
-
- However, local thresholding can cause noise and broken object boundaries.
 - Global thresholding is faster and simpler, but may fail if lighting is uneven.
 - Local thresholding is more flexible but slower and may produce noisy results.
 - Choose global thresholding for simple images and local thresholding for complex, uneven images.

CONCLUSION:

Otsu's global thresholding efficiently segments images with clear intensity differences by finding an optimal single threshold. It is simple and fast but less effective with uneven lighting. Local thresholding adapts to varying illumination, improving segmentation in complex images. However, it may introduce noise and requires more computation.